



AALBORG
UNIVERSITY

Session 1: Introduction

Part 1: What this course is about (and is not about)

Languages and compilers

Hans Hüttel, February 2024

CC BY-NC-SA 4.0 DEED

Attribution-NonCommercial 4.0 International

SPO

Introduction – Languages and compilers

Hans Hüttel

incorporating program examples by Patrick Donnelly and content
by Torben Æ. Mogensen

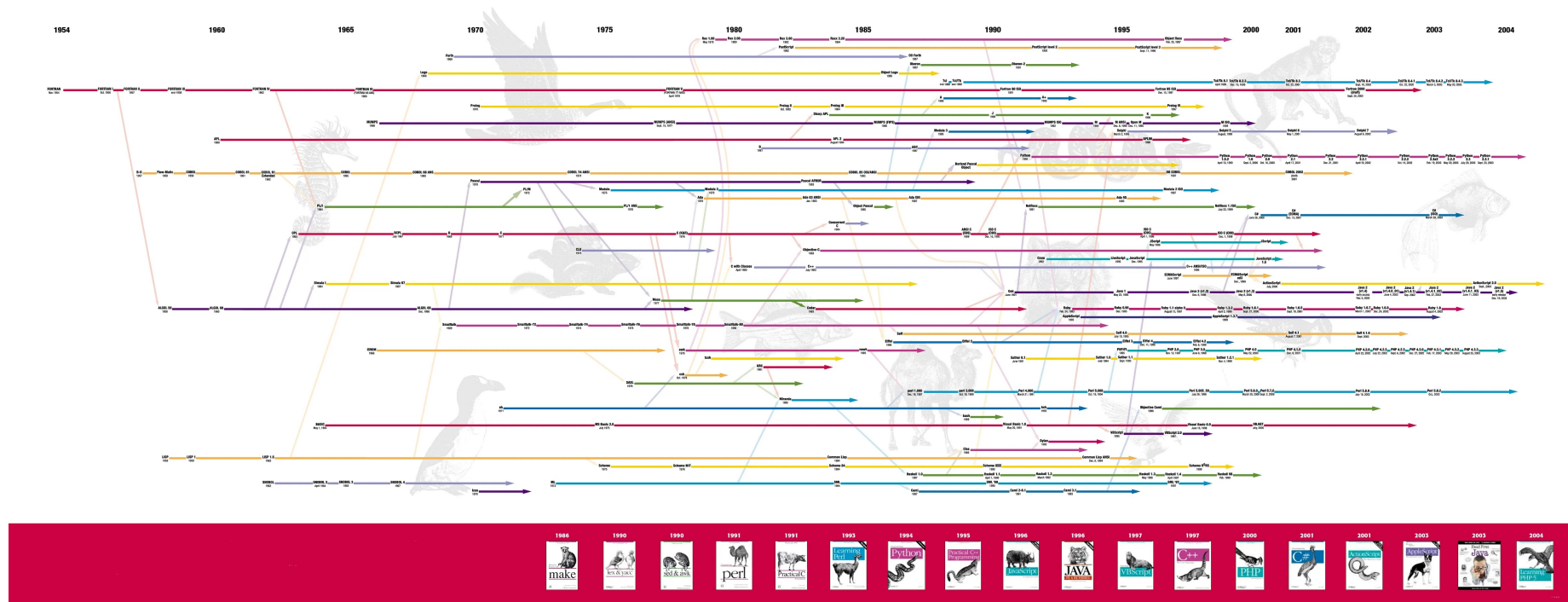
Session 1

Learning goals

The learning goals for this session are

- To understand the notion of programming paradigm
- To be able to explain the fundamental trade-off in programming language design and implementation
- To be able to explain the difference between an interpreter and a compiler
- To understand the notion of abstract syntax
- To be able to define the abstract syntax of a simple language

How many programming languages are there?



The publishing company O'Reilly created a poster that shows the history of programming languages from 1954 to 2004.

Since then, plenty has happened. According to the Online Historical Encyclopaedia of Programming Languages, there are at least **8945** programming languages in existence.

The story never ends

Every year, new programming languages are being created.

Some are experimental and never reach a wider audience.

Others are used in specific application domains (such as statistics, game programming or robotics). We are going to have a look at such **domain-specific languages** later on in this course.

Yet others are **general-purpose languages** that are used for writing programs in a wide variety of contexts.

No textbook can cover every case

- There is no textbook on world literature that summarizes every novel that was ever written.
- There is no textbook on medicine that describes every disease known and all possible treatments for it.
- And in a course about programming languages and their implementation we cannot cover every programming language and every implementation of it.

This is a course about the underlying principles

At university, we teach general theories and principles that can explain the part of the world that we study can and apply to a wide range of situations that one will encounter. The goal is that such general theories and principles can help us understand and help us find good solutions to problems in our subject. All those theories and principles are the results of many years of scientific efforts by clever people.

This is a course about general theories and principles concerning programming languages and their implementation. It should be useful and also show why the theories and principles came about and why we need them.

But what *is* a programming language?

A programming language is a notation that allows us to express algorithms and to express data.

A program is a description of an algorithm (and possibly also data associated with it).

The notation must be **human-readable/writable**. But it must also be **machine-executable**: If we give a computer a program, it will execute the program and in this way carry out the steps of the algorithm.

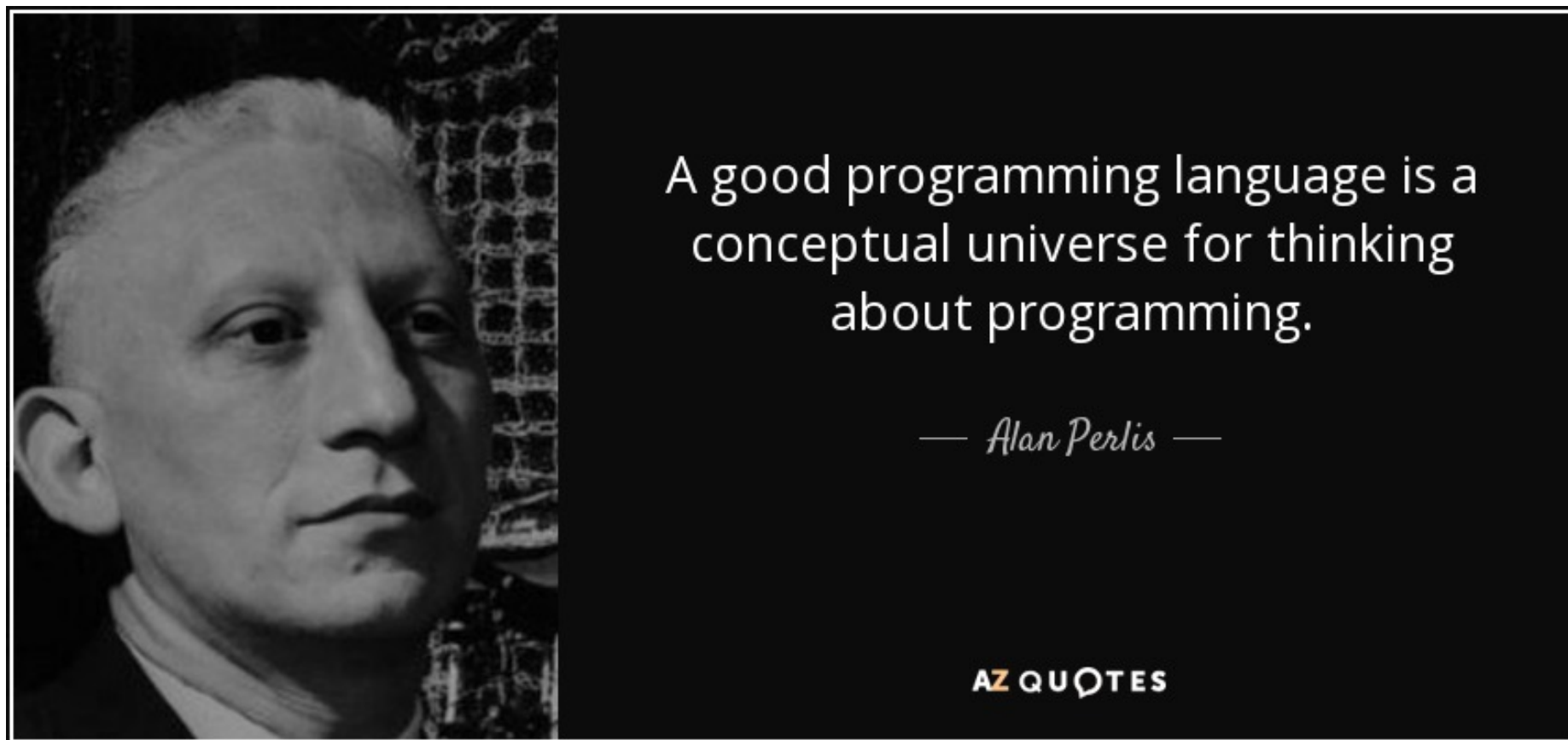
The big trade-off

We must

- find out what is required of our notation if it is supposed to be sensible for humans to read and write – this is **language design**
- find out how to execute a program on a computer (and in an efficient manner) – this is **language implementation**

A well-designed programming language must take both of these concerns into account.

Put very briefly ...



Alan Perlis was an American computer scientist who was involved in the development of the highly influential ALGOL language. He was the first recipient of the ACM Turing Award.

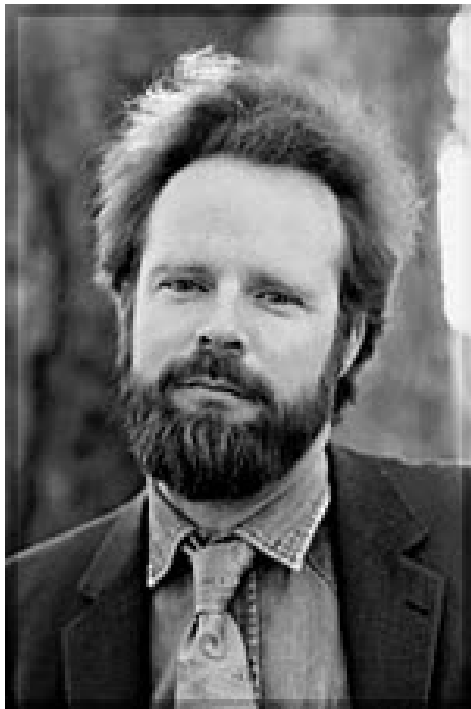
Why are there so many programming languages?

- No programming language is well-suited for every style of programming.
- Every programming language is better suited for expressing certain kinds of algorithms and data structures than others.

The paradigms of programming

Programming paradigms are different ways or styles in which a program can be organized. When you study computing, you must understand and master more than one programming paradigm.

Robert Floyd, professor of computer science at Stanford University, received the 1978 ACM Turing Award.



In his paper *Paradigms of Programming* from that year, he wrote:

If the advancement of the general art of programming requires the continuing invention and elaboration of paradigms, advancement of the art of the individual programmer requires that he expand his repertory of paradigms.

The major programming paradigms

- The imperative paradigm
- The object-oriented paradigm
- The functional paradigm
- The logic paradigm

Every programming language supports one programming paradigm. Many languages support more than one of them.

The object-oriented paradigm should be seen as a sub-paradigm within the imperative paradigm.



AALBORG
UNIVERSITY

Session 1: Introduction

Part 2: The history of programming languages

Languages and compilers

Hans Hüttel, February 2024

CC BY-NC-SA 4.0 DEED

Attribution-NonCommercial 4.0 International

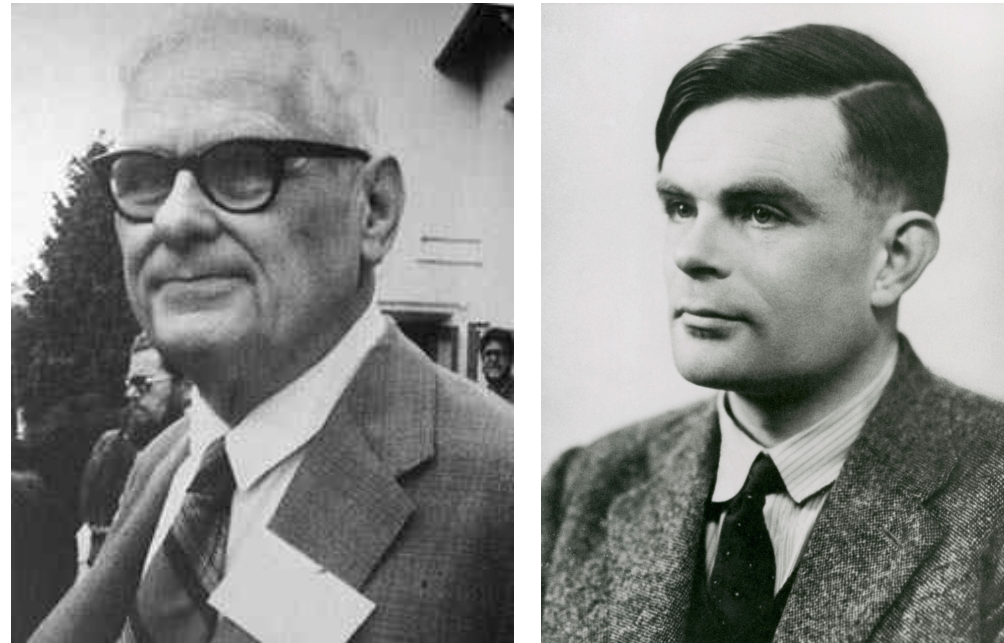
Charles Babbage and Ada Lovelace



The analytical engine was a proposed mechanical general-purpose computer designed by the English mathematician Charles Babbage in 1837. It was never built.

In 1843 the English mathematician Ada Lovelace described how one could use the machine to compute so-called Bernoulli numbers. This was the first description of how a machine could perform the steps of an algorithm – the first program.

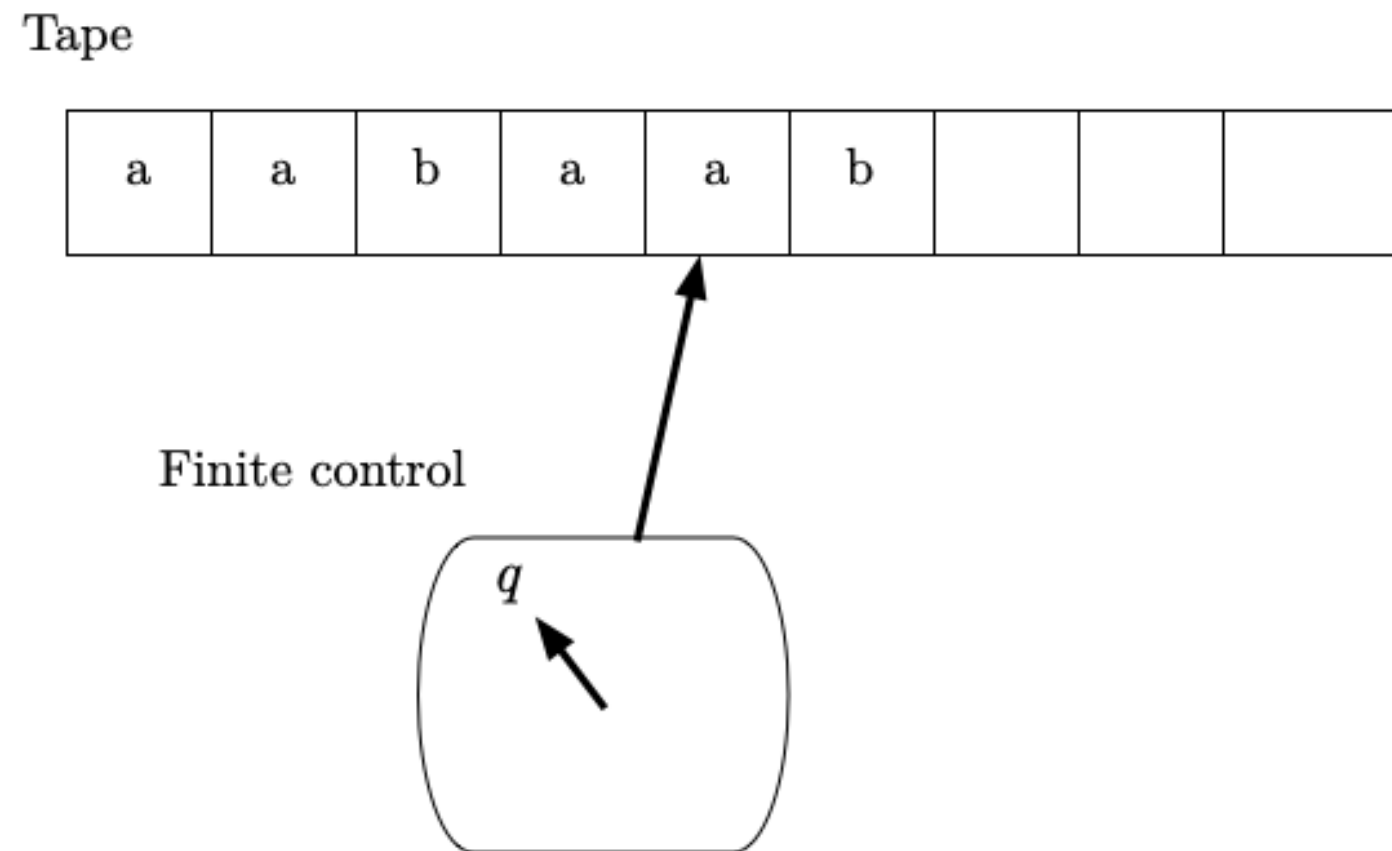
Models of computation in mathematics



In the early part of the 20th century mathematicians wanted to find out if there was a general algorithm that could decide if theorems in mathematics were true or not. Mathematicians looked for a *model of computation* that would be a precise mathematical notion of what an algorithm is.

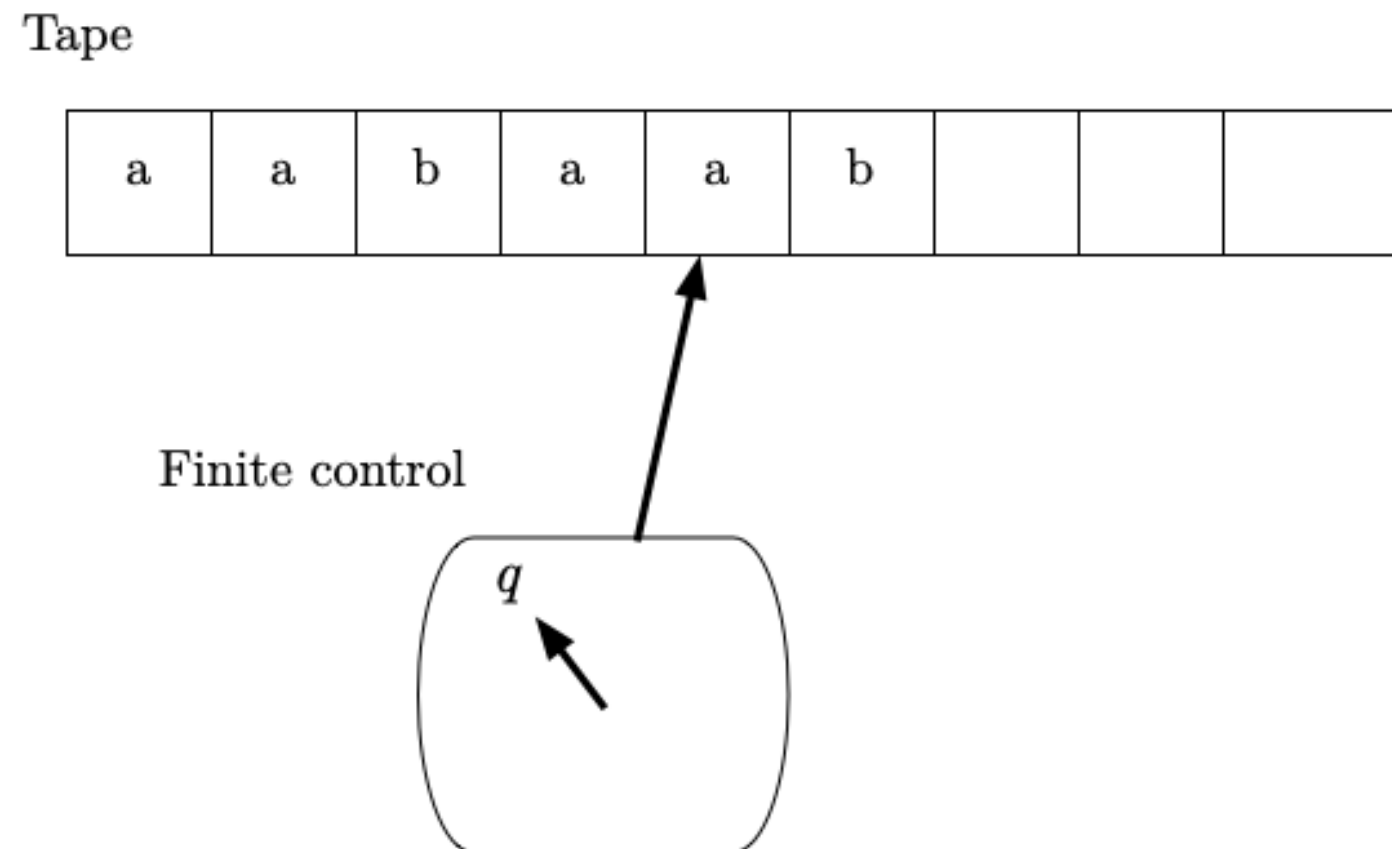
Two of them were Alonzo Church (left) from Princeton University in the United States and Alan Turing from Cambridge University in the United Kingdom.

The Turing machine



In 1936 Alan Turing proved that there is no algorithm for deciding mathematical truth in general. To prove this, he defined a model of computation that we now know as the *Turing machine*.

The Turing machine



A Turing machine has an infinite tape divided into cells. This is the memory of the machine. Each cell can contain a symbol (or nothing). A finite control reads a cell and can then move the tape head left or right, replace the character it read and change its state q .

The λ -calculus

At the same time, Alonzo Church developed a notation for computable functions called the λ -calculus.

Here is an example of what you can write.

$$(\lambda x.xx)(\lambda y.y)$$

He gave a small collection of reduction rules that define how expressions can be simplified.

Church and Turing

Turing proved that Turing machines have the same expressive power as the λ -calculus:

Theorem

*Every algorithm that one can describe as a Turing machine can also be described using an expression in the λ -calculus.
And every algorithm that one can describe using an expression in the λ -calculus can also be described as a Turing machine.*

We say that the λ -calculus is **Turing-complete**.

Later, Turing moved to the United States to work on a PhD with Church as his supervisor.

The first programmable computers

The digital programmable computer was invented more than once around the time of World War Two and became part of the war effort on both sides of the war. This also led to the creation of the first programming languages.

- The Z2 was created by the German engineer Konrad Zuse in 1940.
- The Colossus was invented by Tommy Flowers at Bletchley Park, the British centre of Allied code-breaking during the Second World War.
- The ENIAC was the first electronic programmable computer built in the United States. It performed ballistics trajectory calculations for the U.S. Army.

Plankalkül

The first "real" programming language was designed in 1943 by Konrad Zuse. Plankalkül used programming concepts such as variables, functions, arrays, loops, conditionals and even recursion and filters, but its syntax was *very* different from that of modern programming languages.

$$\begin{array}{l}
 \text{P148} \quad \left| \begin{array}{c} R(V) \Rightarrow R148 \\ V \quad 0 \quad 0 \\ A \quad 5 \quad 0 \end{array} \right. \quad (1) \\
 \\
 \left| \begin{array}{c} x \quad \left[(x \in V) \wedge (x = L0) \right] \Rightarrow Z \\ V \quad 0 \\ K \quad 1 \\ A \quad 4 \quad 5 \quad 3 \quad 4 \end{array} \right. \quad (2) \\
 \\
 \left| \begin{array}{c} (Ex) \quad \left[(x \in V) \wedge R17 \quad (Z, x) \wedge (x = 0) \vee x \right] \\ V \quad 0 \\ K \quad 0 \quad 0 \quad 1 \quad 13 \\ A \quad 4 \quad 4 \quad 5 \quad 2 \quad 2 \quad 3 \quad 0 \end{array} \right. \quad (3) \\
 \\
 \left| \begin{array}{c} \wedge \overline{Ex} \quad \left[(y \in V) \wedge y \wedge R128 \quad (v, y, x) \right] \\ V \quad 0 \\ K \quad 13 \quad 0 \quad 0 \\ A \quad 4 \quad 5 \quad 0 \quad 5 \quad 2 \quad 2 \end{array} \right. \quad (4)
 \end{array}$$

Plankalkül was largely forgotten because of World War Two, and a full implementation was not made until 1972.

This is not the end of the story!

- Many other models of computation exist now. Most of them are known to be Turing-complete. The rest are weaker than Turing machines.
- Alan Turing became a pioneer in computing – he was involved in cryptology during the Second World War and later designed the first digital computer in the United Kingdom.
- The λ -calculus is behind all so-called functional programming languages.

Some influential programming languages

The following is a short list of influential programming languages from the past within different programming paradigms. Many of them are still in use.

It is by no means a complete list but it should give you some idea of where many important concepts come from.

All the language designers that we mention here have received the ACM Turing Award for their work. This is the most prestigious research award in computer science.

FORTRAN

FORTRAN (FORmula TRANslation system) dates back to 1953 and was created at IBM by John Backus and his collaborators. It used a fixed line layout designed for punched cards.

C ← FOR COMMENT		CONTINUATION	FORTRAN STATEMENT	IDENTI- FICATION		
STATEMENT NUMBER						
1	5	6	7	72	73	80
C			PROGRAM FOR FINDING THE LARGEST VALUE			
C		X	ATTAINED BY A SET OF NUMBERS			
			BIGA = A(1)			
			DO 20 I = 2,N			
			IF (BIGA - A(I)) 10, 20, 20			
	10		BIGA = A(I)			
	20		CONTINUE			

FORTRAN used do-loops and three-way jumps as control features and was aimed at numerical computation. Numbers and arrays were the only data types.

gcd in FORTRAN

```
subroutine gcd_iter(value, u, v)
  integer, intent(out) :: value
  integer, intent(inout) :: u, v
  integer :: t
do while( v /= 0 )
  t=u
  u=v
  v = mod(t, v)
enddo
  value = abs(u)
end subroutine gcd_iter
```

gcd in COBOL (part 1)

COBOL was created in 1959 by Jean Sammet and her collaborators and was based on FLOWMATIC designed by Grace Hopper. COBOL was meant for business-oriented applications.

```
IDENTIFICATION DIVISION.
```

```
PROGRAM-ID. GCD.
```

```
DATA DIVISION.
```

```
WORKING-STORAGE SECTION.
```

```
01 A
```

```
01 B
```

```
01 TEMP
```

```
. . .
```

```
PIC 9(10)
```

```
PIC 9(10)
```

```
PIC 9(10)
```

```
VALUE ZEROES.
```

```
VALUE ZEROES.
```

```
VALUE ZEROES.
```


gcd in COBOL (part 2)

PROCEDURE DIVISION.

Begin.

```
DISPLAY "Enter first number, max 10 digits."  
ACCEPT A  
DISPLAY "Enter second number, max 10 digits."  
ACCEPT B  
IF A < B  
    MOVE B TO TEMP  
    MOVE A TO B  
    MOVE TEMP TO B  
END-IF  
PERFORM UNTIL B = 0  
    MOVE A TO TEMP  
    MOVE B TO A  
    DIVIDE TEMP BY B GIVING TEMP REMAINDER B  
END-PERFORM  
DISPLAY "The gcd is " A  
STOP RUN.
```


gcd in Lisp

Lisp was designed by John McCarthy in 1958 and was intended for applications in artificial intelligence. McCarthy was inspired by Church and his work on the λ -calculus and Lisp contains it.

Lisp is the first programming language that supports the functional paradigm.

```
(defun gcd2 (a b)
  (do () ((zerop b) (abs a))
    (shiftf a b (mod a b))))
```

ALGOL60

ALGOL60 was designed by Peter Naur, John Backus, John McCarthy and many others together in 1960. Many important features such as

- the concept of type
- names that can be any length
- multidimensional arrays
- semicolon as a statement separator
- subprogram recursion
- dynamic arrays

gcd in ALGOL60

```
PROC gcd = (INT a, b) INT: (  
  IF a = 0 THEN  
    b  
  ELIF b = 0 THEN  
    a  
  ELIF a > b THEN  
    gcd(b, a MOD b)  
  ELSE  
    gcd(a, b MOD a)  
  FI  
);
```

Simula

Simula was invented in Norway by Ole-Johan Dahl and Kristen Nygaard in 1967. Simula was influenced by ALGOL60 and was intended for simulations.

It is the first language to support the object-oriented paradigm. The Simula language has classes, objects and inheritance – and another important notion called coroutines.

gcd in Simula

Begin

Integer Procedure GCD(M, N); Integer M, N;

Begin

While M<>N do

 If M<N then N := N - M else M := M - N;

GCD := M

End of GCD;

Integer A, B;

OutText("Enter an integer number: ");

OutImage; A := InInt;

OutText("Enter an integer number: ");

OutImage; B := InInt;

OutText("Greatest Common Divisor of your numbers is ");

OutInt(GCD(A,B), 4); OutImage;

End of Program;

gcd in Prolog

Prolog was designed by Alain Colmerauer in 1972 and his collaborators and was intended for applications in natural language processing and artificial intelligence. Prolog is the first logic programming language.

Pattern matching did not originate in Prolog but it is highly important in this language. A program is a collection of logical implications called **clauses** from first-order logic.

```
gcd(X, 0, X):- !.  
gcd(0, X, X):- !.  
gcd(X, Y, D):- X > Y, !, Z is X mod Y, gcd(Y, Z, D).  
gcd(X, Y, D):- Z is Y mod X, gcd(X, Z, D).
```

C

C was designed by Dennis Richie at Bell Labs in 1972. It was originally designed as a systems language supporting Unix.

C has a powerful set of operators (and very poor type checking ...).

C was influenced by ALGOL60 and has in turn influenced many other imperative programming languages.

gcd in C

```
int
gcd_iter(int u, int v) {
    int t;
    while (v) {
        t = u;
        u = v;
        v = t % v;
    }
    return u < 0 ? -u : u; /* abs(u) */
}
```


Standard ML

Standard ML (SML) was designed by Robin Milner and his collaborators in the 1970s. SML supports the functional paradigm. It was originally part of the proof assistant LCF (Logic of Computable Functions) that was used for proving properties of programs.

SML has a very powerful type system with parametric polymorphism and was the first programming language to be given a definition in the form of a structural operational semantics. It has been proved that the type system of SML is safe under the semantics – a well-typed SML cannot exhibit certain run-time errors. This was also a first.

gcd in Standard ML

A program in SML is a collection of function definitions.

```
fun gcd(m,n) = if m=0 then n
                else gcd(n mod m, m);
```



AALBORG
UNIVERSITY

Session 1: Introduction

Part 3: Something about language syntax

Languages and compilers

Hans Hüttel, February 2024

CC BY-NC-SA 4.0 DEED

Attribution-NonCommercial 4.0 International

Understanding syntax

In the following I will try to demonstrate how one can give a precise description of the syntax of a language.

I will give an example of a tiny programming language of a particular kind, namely a so-called imperative programming language (that supports the imperative paradigm).

What is syntax?

The syntax of a language tells us which statements in our language are well-formed and which ones are not. We are only interested in statements that are well-formed.

In English, the sentence

The cat jumps from the wall.

is correct according to the syntax of English, but

Jumps wall cat cat.

is not.

Syntax errors in C

```
int main() {  
int count = 5  
while count ≤ 10 {  
    printf("%d\n", count);  
    count++;  
}  
return 0;  
}
```

There are two syntax errors here that prevents this piece of code from being well formed.

- Missing semicolon after the `count` variable declaration.
- Missing parentheses around the condition in the `while` loop.

Syntax – and beyond

In English, the sentence

The wall jumps from the cat.

is syntactically correct, but it makes no sense – there is a **semantic error** here.

There is more to language than just form, but we first have to consider the aspect of form, that is, syntax.

Expressions

Most programming languages have a notion of **expression**. An expression evaluates to a value. If we let e be an expression, then e could be

- A numeral n
- A variable x
- An addition expression $e_1 + e_2$
- A multiplication expression $e_1 * e_2$

Many programming languages have expressions of this kind – and many others than those shown here. But what about **statements**?

A story about programming language design

There is a reason why so many well-known programming languages have statements.

This has to with the fact that programs have to be machine-executable.

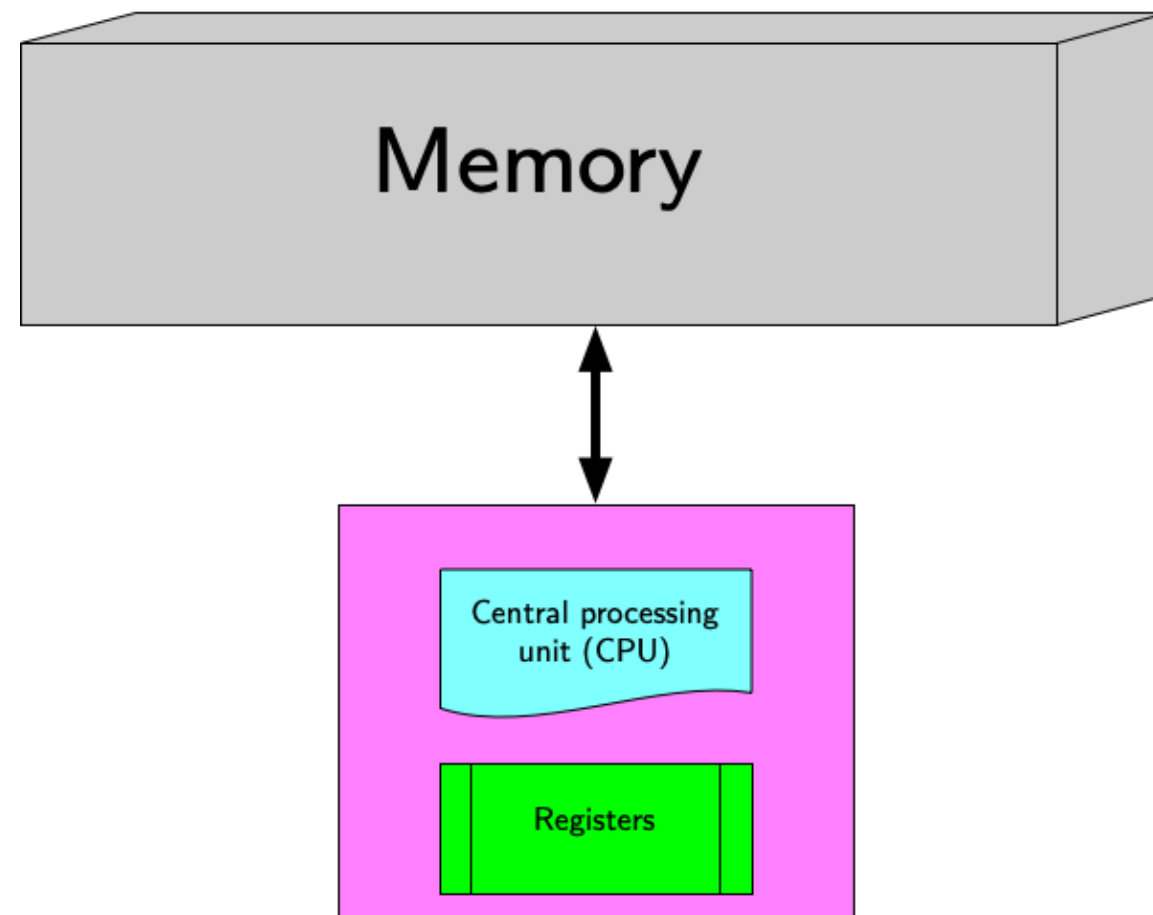
The computer architecture seems to make certain demands (or does it really?)

The von Neumann architecture



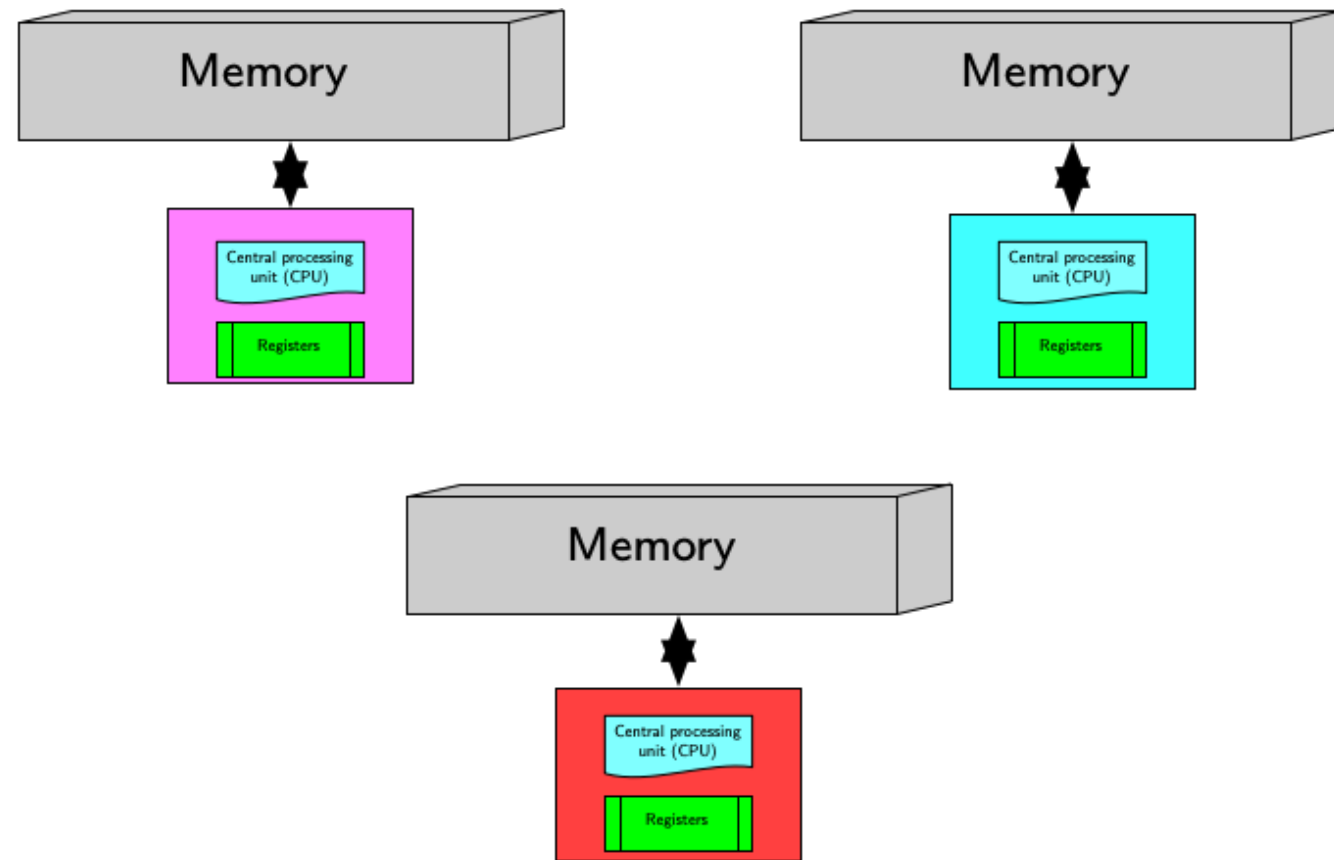
The Hungarian-born mathematician János von Neumann is famous for his many achievements in mathematics and theoretical physics but also for inventing the approach to computer architecture that we still use. During the Second World War, he consulted for the Army's Ballistic Research Laboratory, most notably on the ENIAC project which resulted in the first programmable digital computer.

The von Neumann architecture



A computer has a memory and a central processing unit (CPU) that carries out instructions, reads from and writes to memory.

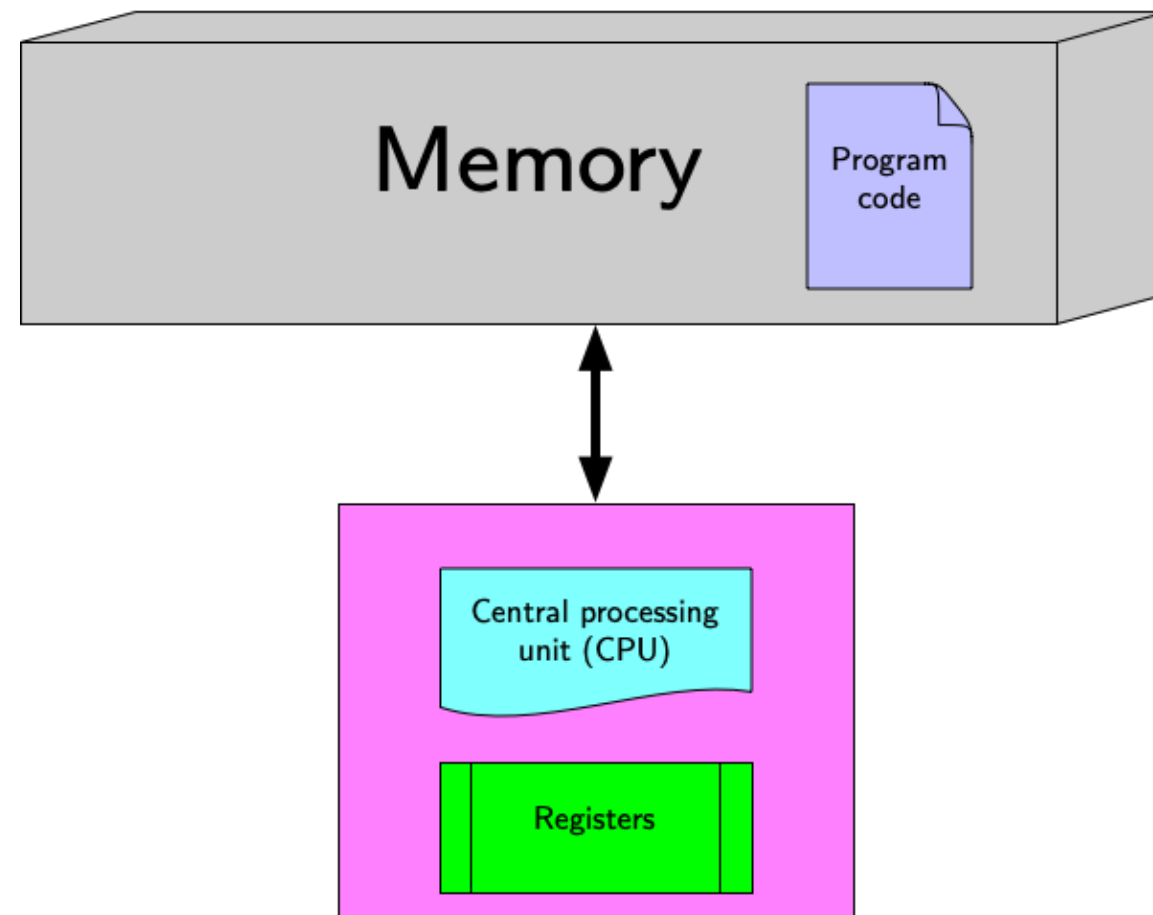
New instructions = new computer!



Originally, one would have change the configuration of the CPU every time one wanted the computer to solve a different task. That is a bit cumbersome.

The idea that von Neumann came up with probably also comes from Turing's work.

The von Neumann architecture



Von Neumann realised that the instructions could be stored in the memory. This way one can keep the configuration of the computer fixed – just store the code in the memory and let the CPU fetch the instructions one by one and then execute them. Now the computer is programmable. **Programs are data, too!**

Imperative programming languages

C, C++, C#, Java, Python and many other well-known languages are called **imperative** languages. Imperative languages have **commands**.

The central command in languages from this programming paradigm is that of assignment:

$$x := e$$

We assign the value of the expression e to the variable x

This command comes directly from the von Neumann architecture.

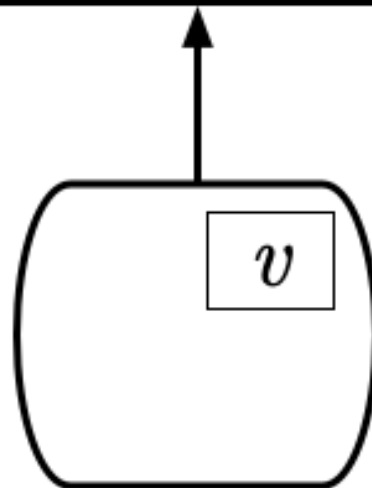
What is an assignment?

$x := e$

Memory



CPU



An assignment $x := e$ evaluates e to some value v , places v in some register, looks up the location of variable x in memory and then writes v to that location.

Control structures

Every command in an imperative language *may* change the value of one or more variables.

Many other constructs in an imperative programming language specify which assignments to carry out and in which order. Let S be any command, then the following are also commands:

- Sequential composition $S_1; S_2$
- If-then-else commands $\text{if } e \text{ then } S_1 \text{ else } S_2$
- Loop commands $\text{while } e \text{ do } S$

What is enough?

The simple imperative programming language, the one that we have just seen here, is Turing-complete!

Every algorithm can be expressed using only the simple imperative commands. This result is called the *structured program theorem*; it was proved by Corrado Bohm and Giuseppe Jacopini in 1966.

But it is cumbersome to program in such a restricted language. A more friendly language would be preferable for actual program development. This is another reason for caring about programming language design.

Syntactic categories

Expressions and commands are what we call **syntactic categories**.

Informally speaking, a syntactic category is a set of entities in a language that all "behave in the same way".

We know the notion of syntactic category from natural languages. In English we have the syntactic categories of nouns, verbs, adjectives and adverbs (and many more).

Formation rules

We can specify that elements of the syntactic category **Exp** of expression have the structure that we describe earlier by a collection of **formation rules**¹. In our case they are

$$e ::= n \mid x \mid e_1 + e_2 \mid e_1 * e_2$$

The formation rules for the syntactic category **Com** of commands are

$$S ::= x := e \mid S_1; S_2 \mid \text{if } e \text{ then } S_1 \text{ else } S_2 \mid \text{while } e \text{ do } S$$

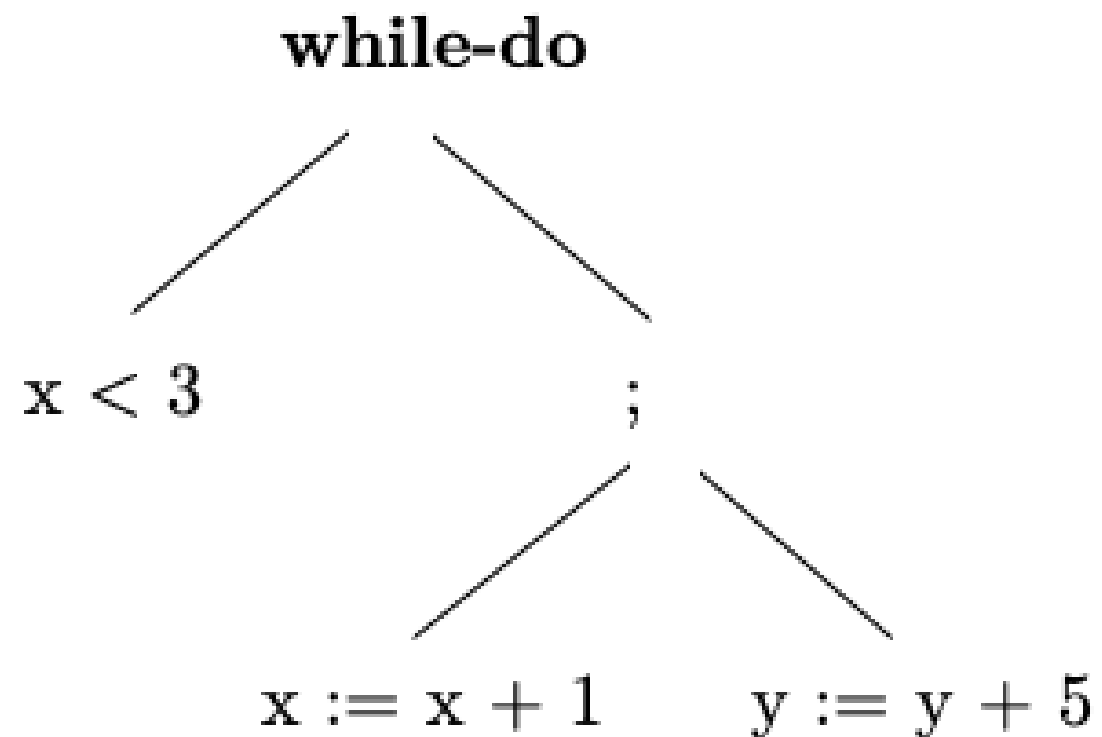
The symbol $::=$ means "can be" and the $\mid$'s separate the choices on the right-hand side. e and S are called **metavariables**. We use indexes such that we can tell different occurrences of them apart.

¹Dansk: opbygningsregler

Abstract syntax trees

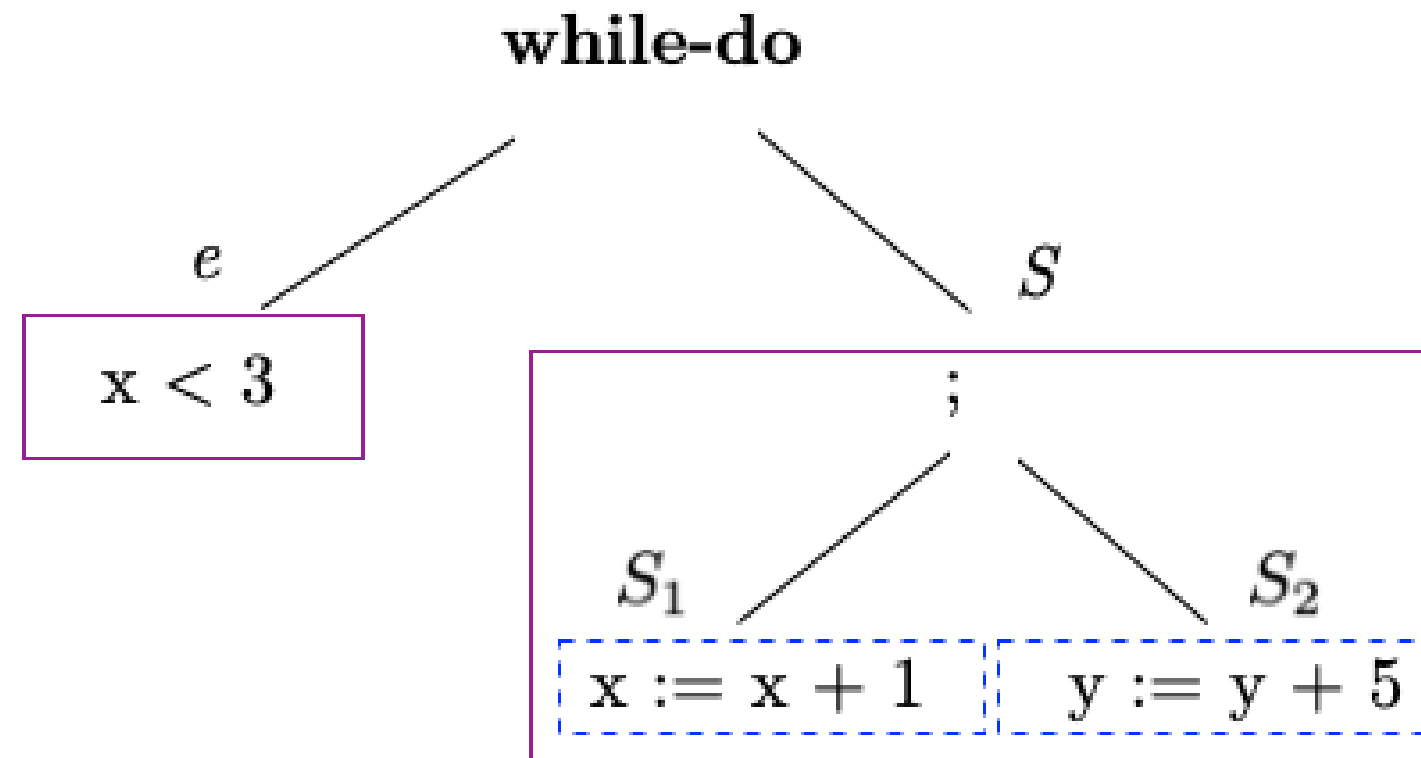
Every element in our syntax can be thought of as a tree. We call trees of this form **abstract syntax trees**.

while $x < 3$ **do** $x := x + 1$; $y := y + 5$



Abstract syntax trees

while $x < 3$ **do** $x := x + 1; y := y + 5$



Here, $x < 3$ and $x := x + 1; y := y + 5$ are the *immediate constituents*² of the while-command. $x := x + 1$ and $y := y + 5$ are the immediate constituents of the sequential composition.

²Dansk: Umiddelbare bestanddele.

Specifying the abstract syntax of a language

- 1 List the syntactic categories of the language.
- 2 For each syntactic category find a name for its metavariable.
- 3 Define a collection of formation rules that define the valid elements of this particular syntactic category.

We call such a syntax an **abstract syntax**. The abstract syntax is the central part of the specification of the syntax of a programming language.

It is needed by the rest of the definition of the language and in the implementation!

Concrete syntax

We tend to think of a program as a text – a string of characters. And that is what a text editor supports.

Therefore we need a piece of software that can transform a syntactically correct program text into the abstract syntax tree.

Such a piece of software is called a **parser**.

And we need another specification of the syntax that sees programs as text. Such a specification is called a **concrete syntax**.

We will return to all this in a later session.

Do we really need to be imperative?

Many computer scientists in the field of programming languages would like to have languages that do not depend on the computer architecture.



One of the first was Peter Landin.

The next 700 programming languages

The Next 700 Programming Languages

P. J. Landin

Univac Division of Sperry Rand Corp., New York, New York

“... today ... 1,700 special programming languages used to ‘communicate’ in over 700 application areas.”—*Computer Software Issues*, an American Mathematical Association Prospectus, July 1965.

A family of unimplemented computing languages is described that is intended to span differences of application area by a unified framework. This framework dictates the rules about the uses of user-coined names, and the conventions about characterizing functional relationships. Within this framework the design of a specific language splits into two independent parts. One is the choice of written appearances of programs (or more generally, their physical representation). The other is the choice of the abstract entities (such as numbers, character-strings, lists of them, functional relations among them) that can be referred to in the language.

The system is biased towards “expressions” rather than “statements.” It includes a nonprocedural (purely functional) subsystem that aims to expand the class of users’ needs that can be met by a single print-instruction, without sacrificing the important properties that make conventional right-hand-side

differences in the set of things provided by the library or operating system. Perhaps had ALGOL 60 been launched as a family instead of proclaimed as a language, it would have fielded some of the less relevant criticisms of its deficiencies.

At first sight the facilities provided in ISWIM will appear comparatively meager. This appearance will be especially misleading to someone who has not appreciated how much of current manuals are devoted to the explanation of common (i.e., problem-orientation independent) logical structure rather than problem-oriented specialties. For example, in almost every language a user can coin names, obeying certain rules about the contexts in which the name is used and their relation to the textual segments that introduce, define, declare, or otherwise constrain its use. These rules vary considerably from one language to

In his paper from 1966, Landin proposed ISWIM (*If You See What I Mean*). This is a functional programming language with higher-order functions and static scope variables. ISWIM was not implemented but later languages captured many of its ideas.

Do we really need to be imperative?

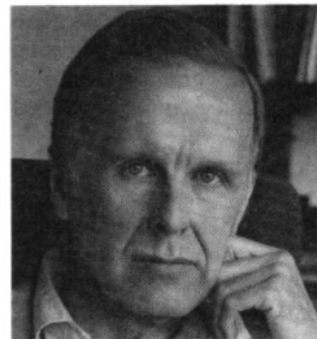


Another early critic of imperative programming was John Backus, the designer behind FORTRAN and ALGOL60.

Backus and functional programming

Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs

John Backus
IBM Research Laboratory, San Jose



General permission to make fair use in teaching or research of all or part of this material is granted to individual readers and to nonprofit libraries acting for them provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery. To otherwise reprint a figure, table, other substantial excerpt, or the entire work requires specific permission as does republication, or systematic or multiple reproduction.

Author's address: 91 Saint Germain Ave., San Francisco, CA 94114.

© 1978 ACM 0001-0782/78/0800-0613 \$00.75

613

Conventional programming languages are growing ever more enormous, but not stronger. Inherent defects at the most basic level cause them to be both fat and weak: their primitive word-at-a-time style of programming inherited from their common ancestor—the von Neumann computer, their close coupling of semantics to state transitions, their division of programming into a world of expressions and a world of statements, their inability to effectively use powerful combining forms for building new programs from existing ones, and their lack of useful mathematical properties for reasoning about programs.

An alternative functional style of programming is founded on the use of combining forms for creating programs. Functional programs deal with structured data, are often nonrepetitive and nonrecursive, are hierarchically constructed, do not name their arguments, and do not require the complex machinery of procedure declarations to become generally applicable. Combining forms can use high level programs to build still higher level ones in a style not possible in conventional languages.

Communications
of
the ACM

August 1978
Volume 21
Number 8

When John Backus received the ACM Turing Award in 1978, he had become opposed to the "von Neumann bottleneck", as he called it. He advocated functional programming with no imperative features.

Variables and their various values

In an imperative language, a variable can have different values within the same context. A simple example is

```
x := 3;  
x := x + 1;  
y := x
```

where `x` has different values in the first and the last line.

This is completely unlike mathematics, as we know it. If we want to solve equations such as

$$x^2 - 2x + 1 = 0$$

the variable x must have the same value everywhere in the equation.

Referential transparency

If a programming language has the property that **every occurrence of a variable in the same context has the same value**, we can think of variables in the same way that we are used to in mathematics. And we can reason about programs the same way we can reason about ordinary mathematical formulas.

A programming language with this nice property has what we call **referential transparency**. This of course means that there can be no notion of assignment.

But is this really possible for a programming language?

No need for assignments!

```
data Tree a = Leaf a | Empty | Node (Tree a) a (Tree a)

insert (Leaf l) x = if l < x then (Node (Leaf l) x Empty)
                      else (Node (Leaf x) l Empty)

insert Empty x = Leaf x

insert (Node t1 root t2) x | x < root = Node t11 root t2
                                where
                                    t11 = insert t1 x

insert (Node t1 root t2) x | x > root = Node t1 root t21
                                where
                                    t21 = insert t2 x

insert (Node t1 root t2) x | x == root = Node t1 root t2
```

Functional programming languages such as Haskell and Clean do not have imperative features at all but have the same power as C, Java etc. Programs in such languages run just fine!



AALBORG
UNIVERSITY

Session 1: Introduction

Part 4: Interpreters and compilers

Languages and compilers

Hans Hüttel, February 2024

CC BY-NC-SA 4.0 DEED

Attribution-NonCommercial 4.0 International

What is the difference?

An **interpreter** is a piece of software that reads and analyzes a program text (the *source code*) and, if the program text is valid, will then execute the program directly.

A **compiler** is a piece of software that read and analyzes a program text (the *source code*) and if the program text is valid, will then generate a *target code* executable. This target code can then executed later on.

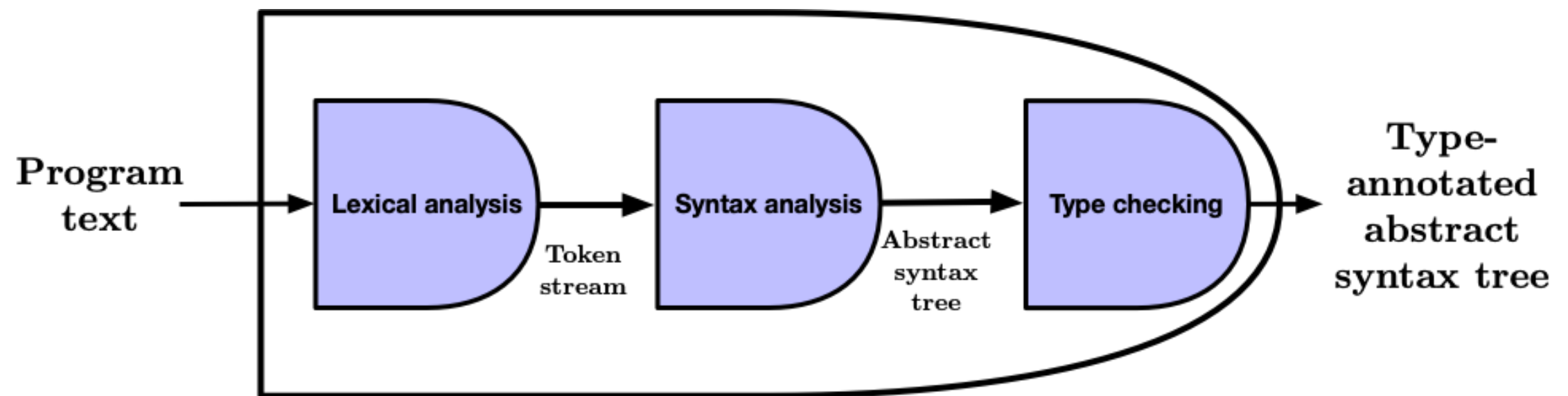
Both approaches have their merits and are widely used.

The front end of an interpreter / a compiler

The front end attempts to build an abstract syntax tree (AST) from a string of characters (the program text).

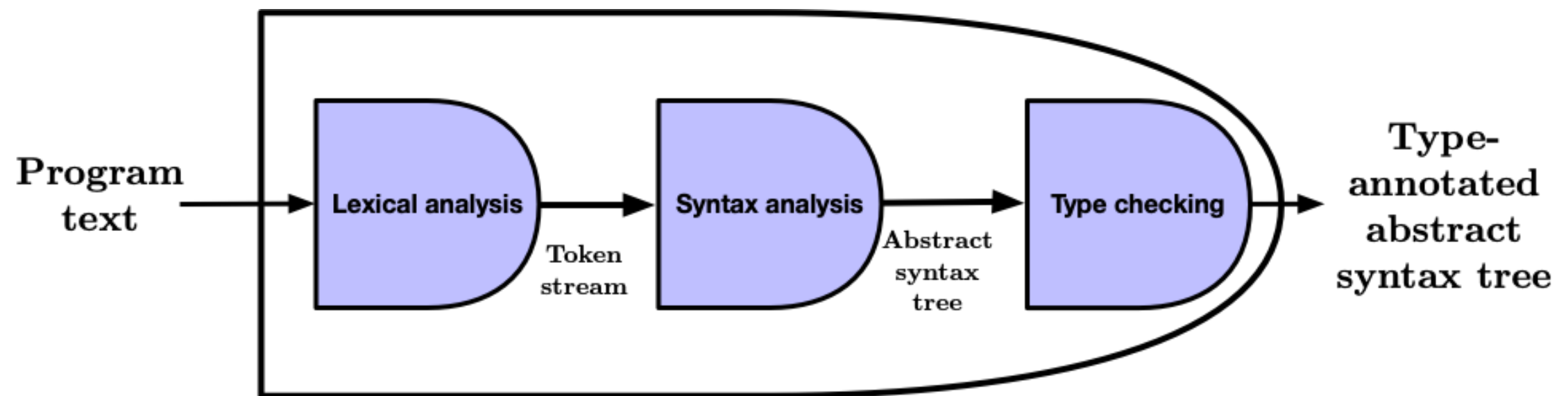
The front end: Lexical analysis

Lexical analysis checks if the individual tokens are well formed, i.e., requires the implementation of a **simple language**. If all is well, this phase returns a stream of tokens.



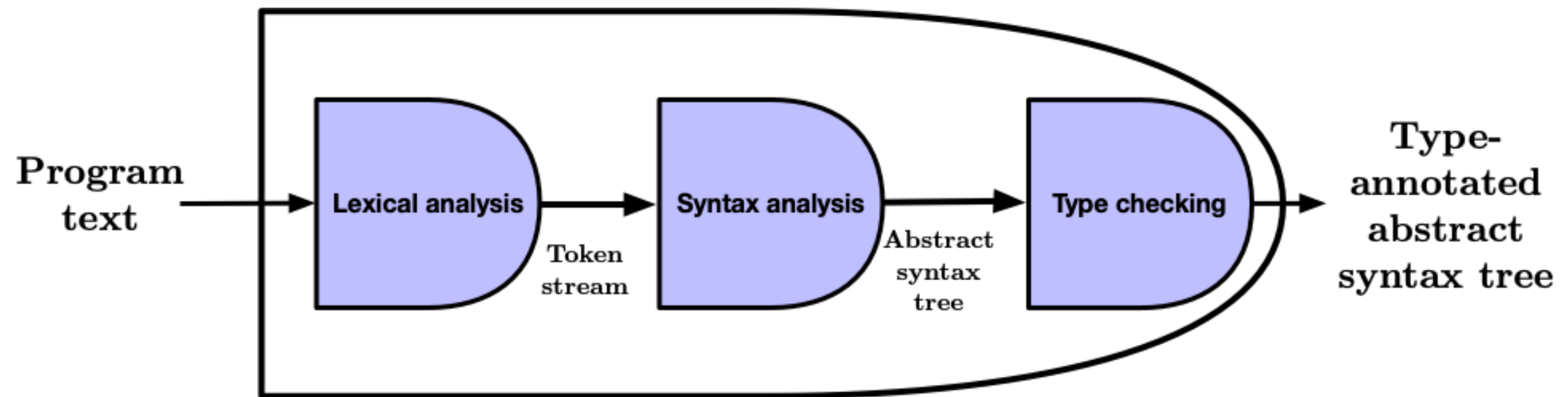
The front end: Syntax analysis

Syntax analysis (aka parsing) checks if the **composition of tokens is well formed**, i.e. complies with the **grammar** of the language. If all is well, this phase returns an AST.

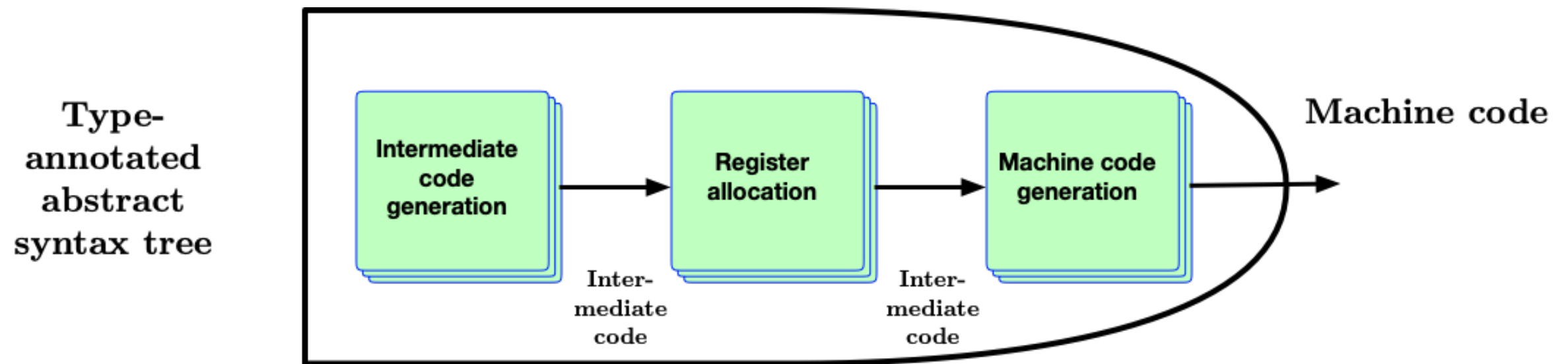


The front end: Type checking

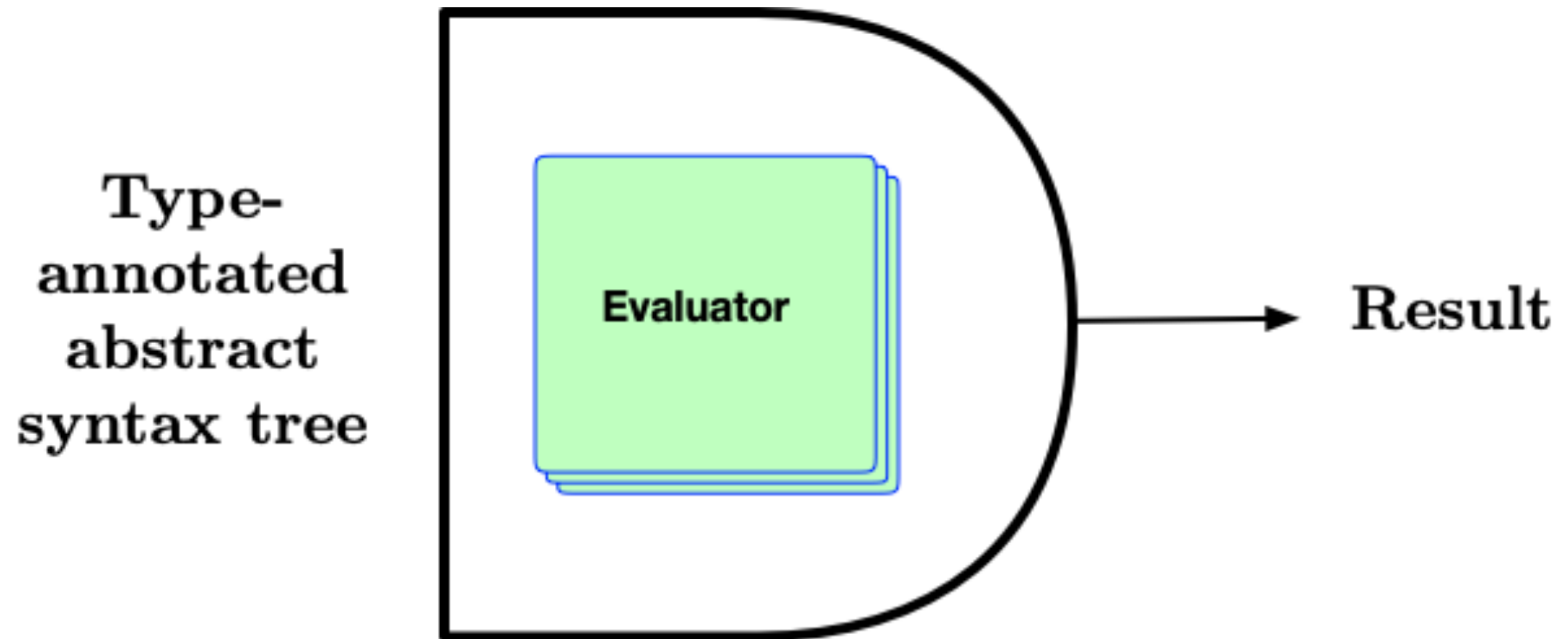
Type checking checks if the program (as represented by the AST) complies with the **type system**. If all is well, it is time for the back end to take over.



The back end of a compiler



The back end of an interpreter



What should we use?

- An interpreter is usually slower than a compiler – even when we do not have to build the syntax tree again. The evaluator part essentially traverses the abstract syntax tree. Compilers generate machine code that also gets optimized.
- On the other hand, it is often much easier to build an interpreter, and if execution speed is not an important issue, interpreters are fine. Moreover, interpreters are much easier to make platform-independent.